Project Report

The final report is the primary document used to assess your project. While the code demonstrates what your system does, the report should explain **why** you made certain design decisions, **how** your system works, and **what** you learned during the project.

- Think of the report as your opportunity to make your case for the project. The report should provide a complete picture of your project. In particular, include all work that is relevant for understanding and assessing your contribution—not only the final successful experiments. If there is work that required substantial effort, influenced the final system, or demonstrates technical or scientific insight, include it—even if it did not end up in the final implementation or produce positive results. **I cannot assess work that is not documented in the report.** The report should guide the assessment; I will not reconstruct the project history solely by inspecting the source code.

  For example:

  - If you spent a considerable amount of time running preliminary experiments that influenced your final system design, report on them.
  - If you invested significant effort (e.g., several weeks) pursuing an approach that ultimately did not become part of your final system, document that work and explain why it was abandoned.
  - Negative results, failed experiments, and design iterations are valuable if they contributed to your understanding of the problem.

- There is no prescribed report format. You are free to choose the structure and style that best fits your project (e.g., a conference-style paper, technical report, or similar. I recommend writing it in Latex or Typst). However, the report should be written in a clear scientific style and contain all information necessary to understand and evaluate your work.
- The report should be at least the equivalent of an 8-page double-column conference paper. Longer reports are welcome if additional content improves the presentation of your work. There is no strict page limit.The focus should be on clarity and completeness rather than on keeping the report short.
- Submit a single .pdf file.
- Use of AI: You are allowed to use AI for writing assistance, e.g. to reformulate paragraphs that you drafted yourself. You are not allowed to use AI to generate the full report.

Expected Contents

Your report should include at least the following sections.

Motivation

- Introduce the problem your agent addresses.
- Explain why the task is interesting or relevant.
- Consider introducing the problem through a concrete example scenario. Use the example to illustrate the type of problem the agent is designed to solve, the challenges involved, and why an agent-based approach is useful.
- Clearly state the goals of your project.

Task Description

Clearly define the task your agent is designed to solve. Describe:

- the input the agent receives,
- the expected output or goal,
- the intended user or use case (if applicable),
- any constraints or requirements of the task.

System Overview

Provide an overview of your agent and its architecture. This should enable a reader to understand how the system works without reading the source code. Figures or diagrams are encouraged whenever they improve clarity.

For example, you may describe:

- the overall workflow,
- the functionality of the agent and its major components,
- how the agent is implemented and configured,
- prompting strategies,
- external tools, MCP servers, or APIs, including their functionality and implementation where applicable,
- memory or planning mechanisms,
- retrieval components,
- important implementation decisions,

- the rationale behind major design choices, including alternatives that were considered and why the final approach was chosen.

Evaluation

Describe how you evaluated your agent. This should include:

- **Evaluation setup:** Describe the overall evaluation scenario, including the tasks, datasets, test cases, or benchmarks used.
- **Evaluation data:** Describe how the evaluation data was obtained, collected, generated, or curated. If you created annotations, describe:
    - the annotation process,
    - annotation guidelines,
    - annotators (e.g., humans, LLMs, other sources),
    - quality control measures,
    - any limitations or biases of the data.
- **Evaluation methodology:** Explain the evaluation procedure and why it is appropriate for measuring your agent's performance.
- **Metrics:** Describe the metrics or criteria used for evaluation.
- **Reproducibility:** Provide sufficient information to reproduce the evaluation (e.g., model versions, relevant hyperparameters, datasets, and evaluation settings). You do not have to include full prompts in the report, but refer to where to find them in the code submission.
- **Comparisons:** If applicable, describe baselines, comparison systems, or ablation experiments.

Results

Present the results of your evaluation.

- **Interpretation of results:** Do not only describe tables and figures; explain what the results show, and what conclusions or limitations can be drawn.
- **Analysis of tables and figures:** Avoid simply repeating their contents. Instead, discuss the most important observations and their implications.
- **Error analysis:** Discuss common failure cases, systematic weaknesses, and insights gained from qualitative analysis of the results.

Discussion

Reflect on your findings and discuss:

- the strengths and limitations of your approach and evaluation methodology,
- challenges encountered during the project,
- any deviations from the original project proposal and the reasons for them,
- possible future improvements and extensions.

Individual Contributions

Include a brief description of the contributions of each team member. The goal is not to provide an exhaustive list of every task, but to give a clear overview of who was primarily responsible for the different aspects of the project (e.g., system design, implementation of specific components, evaluation, annotation, experimentation, report writing, etc.).

If responsibilities evolved over the course of the project, describe the final distribution of work as accurately as possible.

Use of AI

Document how AI tools were used during the project.

Use of AI for writing: Describe how AI tools were used during the writing process and for which purposes (e.g., reformulating paragraphs you drafted, improving clarity, or providing language feedback). AI tools must not be used to generate the full report.

Use of AI for coding: You do not need to identify individual lines of code written with AI assistance. Instead, describe at a higher level:

- which parts or components of the system were developed with support from AI coding tools (e.g., agent components, MCP servers, data processing pipelines, evaluation scripts, interfaces),
- what type of support was provided (e.g., generating code templates, debugging, explaining APIs, suggesting implementations),
- how you reviewed, adapted, and validated the generated code.

Code Submission

The code submission should be provided as a Git repository. The repository should contain all code and resources necessary to understand, run, and evaluate your project.

Repository Requirements

Your repository should include:

- Well-documented code
  - The code should be structured clearly and include comments or documentation where needed.
  - Important design decisions and non-obvious implementation choices should be explained.
- A requirements file
  - Include all required dependencies and their versions where relevant (e.g., `requirements.txt`, `environment.yml`, or equivalent).
  - Do not assume that dependencies are already installed.
- Instructions for running the system
  - Include a README file with clear instructions on how to set up and run your code.
  - The instructions should include:
    - installation steps,
    - environment setup,
    - how to start the agent,
    - example commands,
    - expected inputs and outputs,
    - any additional setup steps required.

The goal is that another person should be able to clone your repository and execute your system by following the provided instructions.

API Keys and External Services

If your project uses cloud-based LLMs or other external services requiring authentication:

- Do not include API keys or credentials in the repository.
- Clearly document:
  - which external services are required,
  - where API keys or credentials need to be provided,
  - how they should be configured (e.g., environment variables, configuration files),
  - any required model names or settings.

Provide example configuration files where useful (without including secrets).

Repository Contents and Versioning

- Include all necessary code and resources required for your final system.
- Do not include unnecessary files such as virtual environments, API keys, large generated files, model weights, or cached data unless they are explicitly required for running the system.
- If there are important experiments, scripts, or components that are not part of the final system but are relevant for understanding your work, include them where appropriate and document their purpose.
- Ensure that the submitted repository corresponds exactly to the version of the system described in your report. If you continue developing after writing the report, clearly identify the final submission version (e.g., using a Git tag or commit hash).